

# Transformer Architectures and Variants

A Mathematical and Architectural Treatment

---

Part 2

Attention and Positional-Encoding Variants

---

# Contents

|          |                                                      |           |
|----------|------------------------------------------------------|-----------|
| <b>1</b> | <b>Why Attention Variants Were Introduced</b>        | <b>1</b>  |
| <b>2</b> | <b>Local and Sparse Attention</b>                    | <b>3</b>  |
| 2.1      | Standard Global Attention . . . . .                  | 3         |
| 2.2      | Sparse Attention . . . . .                           | 3         |
| 2.3      | Local Attention . . . . .                            | 4         |
| 2.4      | Longformer . . . . .                                 | 4         |
| <b>3</b> | <b>Dilated and Structured Sparse Attention</b>       | <b>6</b>  |
| 3.1      | Dilated Attention . . . . .                          | 6         |
| 3.2      | Sparse Transformer . . . . .                         | 6         |
| <b>4</b> | <b>Linear Attention</b>                              | <b>8</b>  |
| 4.1      | The Quadratic Bottleneck . . . . .                   | 8         |
| 4.2      | Kernelized Attention . . . . .                       | 8         |
| 4.3      | Linear Transformer . . . . .                         | 9         |
| <b>5</b> | <b>Performer</b>                                     | <b>10</b> |
| 5.1      | Motivation . . . . .                                 | 10        |
| 5.2      | Kernel Representation . . . . .                      | 10        |
| <b>6</b> | <b>Memory-Efficient Attention</b>                    | <b>12</b> |
| 6.1      | Attention Versus Memory . . . . .                    | 12        |
| 6.2      | FlashAttention . . . . .                             | 12        |
| <b>7</b> | <b>Multi-Query Attention</b>                         | <b>14</b> |
| 7.1      | Standard Multi-Head Attention . . . . .              | 14        |
| 7.2      | Multi-Query Attention . . . . .                      | 14        |
| 7.3      | Why MQA Helps . . . . .                              | 15        |
| <b>8</b> | <b>Grouped-Query Attention</b>                       | <b>16</b> |
| 8.1      | Motivation . . . . .                                 | 16        |
| 8.2      | Mathematical Structure . . . . .                     | 16        |
| <b>9</b> | <b>Relative Positional Representations</b>           | <b>18</b> |
| 9.1      | Limitation of Absolute Position Embeddings . . . . . | 18        |
| 9.2      | Relative Position Attention . . . . .                | 18        |

|                                                     |           |
|-----------------------------------------------------|-----------|
| 9.3 Transformer-XL . . . . .                        | 19        |
| <b>10 Rotary Positional Embeddings</b>              | <b>20</b> |
| 10.1 Motivation . . . . .                           | 20        |
| 10.2 Two-Dimensional Rotation . . . . .             | 20        |
| 10.3 Position-Dependent Rotation . . . . .          | 20        |
| 10.4 Frequency Structure . . . . .                  | 21        |
| <b>11 ALiBi</b>                                     | <b>22</b> |
| 11.1 Motivation . . . . .                           | 22        |
| 11.2 Attention Score . . . . .                      | 22        |
| 11.3 Architectural Change . . . . .                 | 22        |
| <b>12 Attention Variants: Structural Comparison</b> | <b>24</b> |
| <b>13 Implementation References</b>                 | <b>26</b> |
| 13.1 Longformer . . . . .                           | 26        |
| 13.2 Sparse Transformer . . . . .                   | 26        |
| 13.3 Linear Transformer . . . . .                   | 26        |
| 13.4 Performer . . . . .                            | 26        |
| 13.5 FlashAttention . . . . .                       | 27        |
| 13.6 Multi-Query Attention . . . . .                | 27        |
| 13.7 Grouped-Query Attention . . . . .              | 27        |
| 13.8 Transformer-XL . . . . .                       | 27        |
| 13.9 RoPE . . . . .                                 | 27        |
| 13.10ALiBi . . . . .                                | 27        |



# Chapter 1

## Why Attention Variants Were Introduced

The standard Transformer attention operation is

$$\text{Attention}(Q, K, V) = \text{softmax} \left( \frac{QK^T}{\sqrt{d_k}} \right) V. \quad (1.1)$$

For a sequence of length  $n$ ,

$$QK^T \in \mathbb{R}^{n \times n}. \quad (1.2)$$

Consequently, standard self-attention requires interactions between every pair of positions.

Its dominant attention computation scales as

$$O(n^2 d_k), \quad (1.3)$$

and the attention matrix requires

$$O(n^2) \quad (1.4)$$

memory.

This becomes problematic when  $n$  is large.

Transformer research therefore developed several different directions.

The attention operation can be modified by:

1. restricting which positions interact;
2. approximating the attention matrix;
3. changing how keys and values are represented;
4. changing how attention scores are constructed;
5. changing the positional information supplied to attention;
6. replacing absolute positional embeddings with relative positional mechanisms;
7. introducing position-dependent transformations directly into  $Q$  and  $K$ .

These modifications produce substantially different Transformer families.

## Chapter 2

# Local and Sparse Attention

### 2.1 Standard Global Attention

For standard self-attention,

$$H \in \mathbb{R}^{n \times d_{\text{model}}} \quad (2.1)$$

is projected into

$$Q = HW^Q, \quad K = HW^K, \quad V = HW^V. \quad (2.2)$$

The complete pairwise score matrix is

$$S = QK^T. \quad (2.3)$$

Every position can therefore potentially interact with every other position.

### 2.2 Sparse Attention

Sparse attention introduces a structural constraint on the allowed interactions.

Define an attention mask

$$M \in \mathbb{R}^{n \times n}. \quad (2.4)$$

For an allowed interaction,

$$M_{ij} = 0, \quad (2.5)$$

while for a forbidden interaction,

$$M_{ij} = -\infty. \quad (2.6)$$

The attention becomes

$$A = \text{softmax} \left( \frac{QK^T}{\sqrt{d_k}} + M \right). \quad (2.7)$$

The output remains

$$Z = AV, \quad (2.8)$$

but only selected entries of the attention matrix can become nonzero.

## 2.3 Local Attention

Suppose each position can attend only to positions within a window of radius  $w$ .

Then position  $i$  attends to

$$j \in [\max(1, i - w), \min(n, i + w)]. \quad (2.9)$$

The mask becomes

$$M_{ij} = \begin{cases} 0, & |i - j| \leq w, \\ -\infty, & |i - j| > w. \end{cases} \quad (2.10)$$

Therefore,

$$A_{ij} = 0 \quad \text{when} \quad |i - j| > w. \quad (2.11)$$

The number of interactions is approximately

$$O(nw) \quad (2.12)$$

rather than

$$O(n^2). \quad (2.13)$$

The motivation is that many applications contain strong short-range dependencies.

## 2.4 Longformer

Longformer introduced a sparse attention pattern designed for long documents.

Paper:

*Longformer: The Long-Document Transformer*

The central idea is to combine different types of attention patterns rather than performing complete global attention.

A token can receive local attention within a sliding window.

Selected positions can additionally receive global attention.

Let the local neighborhood of position  $i$  be

$$\mathcal{N}(i). \quad (2.14)$$

Then local attention is computed only over



$$j \in \mathcal{N}(i). \quad (2.15)$$

For selected global positions, the model additionally permits

$$j \in \mathcal{G}, \quad (2.16)$$

where  $\mathcal{G}$  is the set of global positions.

The resulting sparsity pattern is therefore not simply a fixed local window.

It combines

$$\text{local attention} + \text{global attention}. \quad (2.17)$$

This allows information to propagate locally through the sequence while selected tokens provide long-range communication.

Repository:

<https://github.com/allenai/longformer>

## Chapter 3

# Dilated and Structured Sparse Attention

### 3.1 Dilated Attention

A local window restricts attention to neighboring positions.

A dilated pattern instead introduces gaps.

For dilation factor  $d$ , a position may attend to

$$i - d, i, i + d, \tag{3.1}$$

or more generally

$$i + kd, \quad k \in \mathbb{Z}, \tag{3.2}$$

subject to sequence boundaries.

The purpose is to increase the receptive field without restoring full quadratic attention.

### 3.2 Sparse Transformer

The Sparse Transformer approach introduced structured sparse attention patterns.

Paper:

*Generating Long Sequences with Sparse Transformers*

Instead of constructing all

$$n^2 \tag{3.3}$$

pairwise interactions, the sequence is partitioned according to structured attention patterns.

The general form remains

$$A = \text{softmax} \left( \frac{QK^T}{\sqrt{d_k}} + M \right), \tag{3.4}$$

but  $M$  is constructed so that only selected entries are accessible.

The key architectural modification is therefore not the attention formula itself, but the connectivity pattern encoded by  $M$ .

## Chapter 4

# Linear Attention

### 4.1 The Quadratic Bottleneck

Standard attention is

$$Z = \text{softmax}(QK^T)V. \quad (4.1)$$

Ignoring scaling for the moment, the expensive part is the construction of

$$QK^T \in \mathbb{R}^{n \times n}. \quad (4.2)$$

Linear-attention methods attempt to avoid explicitly constructing this matrix.

### 4.2 Kernelized Attention

Suppose the attention similarity can be expressed using a feature map

$$\phi(\cdot). \quad (4.3)$$

A kernel can be written as

$$k(q, k) = \phi(q)^T \phi(k). \quad (4.4)$$

The unnormalized attention output for query  $q_i$  can then be written as

$$z_i = \sum_{j=1}^n \phi(q_i)^T \phi(k_j) v_j. \quad (4.5)$$

Using associativity,

$$z_i = \phi(q_i)^T \left( \sum_{j=1}^n \phi(k_j) v_j^T \right). \quad (4.6)$$

The quantity

$$S = \sum_{j=1}^n \phi(k_j) v_j^T \quad (4.7)$$

can be computed once.

Likewise,

$$z_i = \frac{\phi(q_i)^T S}{\phi(q_i)^T \sum_{j=1}^n \phi(k_j)}. \quad (4.8)$$

This changes the computation from explicitly constructing an  $n \times n$  matrix to accumulating quantities over the sequence.

### 4.3 Linear Transformer

Paper:

*Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention*

The central observation is that certain attention formulations can be rearranged so that the summation over sequence positions occurs before multiplication by the query.

Instead of

$$(QK^T)V, \quad (4.9)$$

one can exploit associativity:

$$Q(K^T V). \quad (4.10)$$

Because

$$K^T V \quad (4.11)$$

has dimensions independent of  $n \times n$ , the intermediate attention matrix does not have to be constructed.

This is particularly useful for autoregressive processing.

# Chapter 5

## Performer

### 5.1 Motivation

Performer was introduced to approximate softmax attention with linear complexity.

Paper:

*Rethinking Attention with Performers*

The standard attention kernel is

$$\exp(q^T k). \quad (5.1)$$

The Performer uses a random feature representation to approximate this kernel.

### 5.2 Kernel Representation

The goal is to obtain a feature map  $\phi$  satisfying an approximation of the form

$$\exp(q^T k) \approx \phi(q)^T \phi(k). \quad (5.2)$$

Then the attention computation can be rewritten as

$$z_i = \frac{\sum_{j=1}^n \phi(q_i)^T \phi(k_j) v_j}{\sum_{j=1}^n \phi(q_i)^T \phi(k_j)}. \quad (5.3)$$

The numerator can be rearranged:

$$\sum_{j=1}^n \phi(q_i)^T \phi(k_j) v_j = \phi(q_i)^T \left( \sum_{j=1}^n \phi(k_j) v_j^T \right). \quad (5.4)$$

The denominator becomes

$$\phi(q_i)^T \left( \sum_{j=1}^n \phi(k_j) \right). \quad (5.5)$$

Therefore,

$$z_i = \frac{\phi(q_i)^T \left( \sum_j \phi(k_j) v_j^T \right)}{\phi(q_i)^T \left( \sum_j \phi(k_j) \right)}. \quad (5.6)$$

The quadratic attention matrix is avoided.

Repository:

[Performer implementation](#)

## Chapter 6

# Memory-Efficient Attention

### 6.1 Attention Versus Memory

A distinction must be made between reducing arithmetic complexity and reducing memory consumption.

Standard attention creates

$$A \in \mathbb{R}^{n \times n}. \quad (6.1)$$

Even when the mathematical operation remains equivalent to standard attention, the implementation can avoid materializing the full matrix.

This produces a different class of optimization from sparse or approximate attention.

### 6.2 FlashAttention

Paper:

*FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*

FlashAttention does not replace softmax attention with an approximation.

The mathematical operation remains

$$O = \text{softmax} \left( \frac{QK^T}{\sqrt{d_k}} \right) V. \quad (6.2)$$

The change occurs in how the computation is performed.

Instead of materializing the complete matrix

$$QK^T, \quad (6.3)$$

the computation is divided into blocks.

Suppose

$$Q = \begin{bmatrix} Q_1 \\ Q_2 \\ \vdots \end{bmatrix}, \quad K = \begin{bmatrix} K_1 \\ K_2 \\ \vdots \end{bmatrix}, \quad V = \begin{bmatrix} V_1 \\ V_2 \\ \vdots \end{bmatrix}. \quad (6.4)$$



Attention scores are computed for blocks rather than storing the entire score matrix. The softmax computation is performed using numerically stable online normalization. For a row with scores  $s_1, \dots, s_n$ , ordinary softmax uses

$$p_j = \frac{e^{s_j}}{\sum_r e^{s_r}}. \quad (6.5)$$

A stable implementation maintains a running maximum and running normalization factor so that the complete  $n \times n$  matrix does not need to reside in memory.

The mathematical result remains exact up to floating-point arithmetic.

Repository:

[FlashAttention](#)

The architectural significance is therefore different from linear attention:

$$\text{FlashAttention} \neq \text{approximate attention}. \quad (6.6)$$

Instead,

$$\text{FlashAttention} = \text{exact attention with a different computation strategy}. \quad (6.7)$$

## Chapter 7

# Multi-Query Attention

### 7.1 Standard Multi-Head Attention

In standard multi-head attention, every head has its own

$$Q_h, \quad K_h, \quad V_h. \quad (7.1)$$

Thus, for  $H$  heads,

$$K_1, \dots, K_H \quad (7.2)$$

and

$$V_1, \dots, V_H \quad (7.3)$$

must be stored.

This can become expensive during autoregressive generation, where key/value representations from previous tokens are cached.

### 7.2 Multi-Query Attention

Multi-Query Attention (MQA) keeps multiple query heads but shares the keys and values.

Paper:

*Fast Transformer Decoding: One Write-Head is All You Need*

Instead of

$$Q_h = HW_h^Q, \quad (7.4)$$

$$K_h = HW_h^K, \quad (7.5)$$

$$V_h = HW_h^V, \quad (7.6)$$

for every head, MQA computes

$$Q_h = HW_h^Q \quad (7.7)$$

for every query head, but uses shared

$$K = HW^K \quad (7.8)$$

and

$$V = HW^V. \quad (7.9)$$

Each query head therefore computes

$$Z_h = \text{softmax} \left( \frac{Q_h K^T}{\sqrt{d_k}} \right) V. \quad (7.10)$$

The queries remain multi-headed while the keys and values are shared.

### 7.3 Why MQA Helps

During autoregressive decoding, previously computed keys and values are stored in the KV cache.

With standard multi-head attention, the cache scales with the number of heads.

With MQA, the same keys and values are reused by all query heads.

Therefore, the KV cache is substantially smaller.

The fundamental modification is

$$\boxed{\text{many } Q \text{ heads} + \text{one } K, V \text{ head}}. \quad (7.11)$$

## Chapter 8

# Grouped-Query Attention

### 8.1 Motivation

Multi-Query Attention reduces the KV cache by sharing keys and values across all query heads.

However, complete sharing can reduce representational capacity.

Grouped-Query Attention (GQA) provides an intermediate design.

Paper:

*GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints*

Suppose there are

$$H \tag{8.1}$$

query heads and

$$G \tag{8.2}$$

key/value groups, where

$$G \leq H. \tag{8.3}$$

Each group of query heads shares one key/value head.

### 8.2 Mathematical Structure

For query head  $h$ ,

$$Q_h = HW_h^Q. \tag{8.4}$$

Suppose query heads are divided into groups.

For group  $g$ ,

$$K_g = HW_g^K, \tag{8.5}$$

and

$$V_g = HW_g^V. \quad (8.6)$$

Every query head assigned to group  $g$  uses the same  $K_g, V_g$ .

Thus,

$$Z_h = \text{softmax} \left( \frac{Q_h K_{g(h)}^T}{\sqrt{d_k}} \right) V_{g(h)}. \quad (8.7)$$

Here,

$$g(h) \quad (8.8)$$

maps query head  $h$  to its key/value group.

The three configurations are therefore

$$G = H \quad \Rightarrow \quad \text{Multi-Head Attention}, \quad (8.9)$$

$$1 < G < H \quad \Rightarrow \quad \text{Grouped-Query Attention}, \quad (8.10)$$

and

$$G = 1 \quad \Rightarrow \quad \text{Multi-Query Attention}. \quad (8.11)$$

This provides a continuous architectural trade-off between independent KV heads and complete KV sharing.

## Chapter 9

# Relative Positional Representations

### 9.1 Limitation of Absolute Position Embeddings

The original Transformer adds a position-dependent vector to the token representation:

$$H^{(0)} = E_X + PE. \quad (9.1)$$

The position is therefore encoded directly into the input representation.

An alternative is to make the attention mechanism explicitly depend on relative distance.

For positions  $i$  and  $j$ , define

$$r = i - j. \quad (9.2)$$

The model can then use a learned or parameterized representation

$$a_r \quad (9.3)$$

associated with the relative displacement.

### 9.2 Relative Position Attention

A generic relative attention score can be written as

$$s_{ij} = \frac{q_i^T k_j}{\sqrt{d_k}} + b_{i-j}. \quad (9.4)$$

The attention weights become

$$A_{ij} = \frac{\exp(s_{ij})}{\sum_r \exp(s_{ir})}. \quad (9.5)$$

Therefore,

$$Z_i = \sum_j A_{ij} v_j. \quad (9.6)$$

The positional information enters the attention score itself rather than only being added to the input embeddings.

### 9.3 Transformer-XL

Transformer-XL introduced relative positional representations together with a recurrence mechanism for extending effective context.

Paper:

*Transformer-XL: Attentive Language Models Beyond a Fixed-Length Context*

Let the current segment representation be

$$H_\tau. \tag{9.7}$$

A previous segment representation can be reused through a memory

$$M_{\tau-1}. \tag{9.8}$$

The effective key/value sequence can therefore contain

$$[M_{\tau-1}, H_\tau]. \tag{9.9}$$

The current query representations are computed from the current segment, while keys and values can include cached information from previous segments.

The architecture therefore avoids recomputing all previous hidden states for every new segment.

## Chapter 10

# Rotary Positional Embeddings

### 10.1 Motivation

Rotary Position Embedding (RoPE) introduces positional information by rotating query and key vectors according to their positions.

Paper:

*RoFormer: Enhanced Transformer with Rotary Position Embedding*

The important feature is that positional information is inserted directly into the query and key representations.

### 10.2 Two-Dimensional Rotation

Consider a two-dimensional pair

$$x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}. \quad (10.1)$$

A rotation through angle  $\theta$  is

$$R(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}. \quad (10.2)$$

The rotated representation is

$$x' = R(\theta)x. \quad (10.3)$$

RoPE applies this operation independently to pairs of dimensions.

### 10.3 Position-Dependent Rotation

For position  $m$ , let

$$R_m \quad (10.4)$$

be the corresponding block-diagonal rotation matrix.



The query and key become

$$q'_m = R_m q_m, \quad (10.5)$$

and

$$k'_n = R_n k_n. \quad (10.6)$$

The attention score is then

$$(q'_m)^T k'_n. \quad (10.7)$$

Because rotation matrices satisfy

$$R_m^T R_n = R_{n-m}, \quad (10.8)$$

the inner product can be expressed as

$$(q'_m)^T k'_n = q_m^T R_m^T R_n k_n = q_m^T R_{n-m} k_n. \quad (10.9)$$

Thus the interaction depends naturally on the relative displacement

$$n - m. \quad (10.10)$$

This is a major distinction from simply adding an absolute position vector to the token embedding.

## 10.4 Frequency Structure

For a pair of dimensions, a position-dependent angle can be defined as

$$\theta_{m,i} = m\omega_i, \quad (10.11)$$

where

$$\omega_i \quad (10.12)$$

is a dimension-dependent frequency.

The resulting rotation is

$$R_{m,i} = \begin{bmatrix} \cos(m\omega_i) & -\sin(m\omega_i) \\ \sin(m\omega_i) & \cos(m\omega_i) \end{bmatrix}. \quad (10.13)$$

Different dimension pairs therefore encode position using different frequencies.

# Chapter 11

## ALiBi

### 11.1 Motivation

ALiBi introduces a simple positional bias directly into the attention scores.

Paper:

*Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation*

Instead of adding a positional embedding to the input, ALiBi modifies the attention score.

### 11.2 Attention Score

For query position  $i$  and key position  $j$ , the standard score is

$$s_{ij} = \frac{q_i^T k_j}{\sqrt{d_k}}. \quad (11.1)$$

ALiBi modifies this to

$$s_{ij} = \frac{q_i^T k_j}{\sqrt{d_k}} - m_h |i - j|, \quad (11.2)$$

where  $m_h$  is a head-specific slope.

The resulting attention is

$$A_{ij} = \text{softmax}_j \left( \frac{q_i^T k_j}{\sqrt{d_k}} - m_h |i - j| \right). \quad (11.3)$$

The farther a key is from the query, the larger the negative bias.

Consequently, the mechanism naturally favors nearby positions unless the content-based attention score compensates for the distance penalty.

### 11.3 Architectural Change

The original Transformer uses

$$H^{(0)} = E_X + PE. \quad (11.4)$$

ALiBi instead leaves the token representation without an explicit learned positional embedding and introduces positional information through the attention logits:

$$QK^T \rightarrow QK^T + B. \quad (11.5)$$

Thus the positional mechanism acts directly on the pairwise interaction between positions.

## Chapter 12

# Attention Variants: Structural Comparison

The major attention modifications introduced in this part can be understood by identifying which component of the standard attention computation changes.

The baseline is

$$A = \text{softmax} \left( \frac{QK^T}{\sqrt{d_k}} \right), \quad Z = AV. \quad (12.1)$$

Sparse attention modifies the connectivity:

$$A = \text{softmax} \left( \frac{QK^T}{\sqrt{d_k}} + M \right). \quad (12.2)$$

Linear attention modifies the computation so that the  $n \times n$  attention matrix need not be explicitly formed.

Performer approximates the softmax kernel using a feature map:

$$\exp(q^T k) \approx \phi(q)^T \phi(k). \quad (12.3)$$

FlashAttention retains the mathematical attention function but changes its implementation to reduce memory movement and avoid materializing the complete attention matrix.

MQA changes the number of independent key/value projections:

$$H \text{ query heads} \rightarrow 1 \text{ KV head}. \quad (12.4)$$

GQA generalizes this:

$$H \text{ query heads} \rightarrow G \text{ KV groups}, \quad 1 < G < H. \quad (12.5)$$

Relative positional attention changes the score:

$$s_{ij} = \frac{q_i^T k_j}{\sqrt{d_k}} + b_{i-j}. \quad (12.6)$$

RoPE rotates the query and key vectors:

$$q_i \rightarrow R_i q_i, \quad k_j \rightarrow R_j k_j. \quad (12.7)$$

ALiBi adds a distance-dependent bias:

$$s_{ij} = \frac{q_i^T k_j}{\sqrt{d_k}} - m_h |i - j|. \quad (12.8)$$

These represent fundamentally different forms of architectural modification.

## Chapter 13

# Implementation References

The principal papers and implementations discussed in this part are collected here for direct reference.

### 13.1 Longformer

Paper:

[Longformer: The Long-Document Transformer](#)

Code:

<https://github.com/allenai/longformer>

### 13.2 Sparse Transformer

Paper:

[Generating Long Sequences with Sparse Transformers](#)

### 13.3 Linear Transformer

Paper:

[Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention](#)

### 13.4 Performer

Paper:

[Rethinking Attention with Performers](#)

Code:

<https://github.com/google-research/google-research/tree/master/performer>

## 13.5 FlashAttention

Paper:

[FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness](#)

Code:

<https://github.com/Dao-AI-Lab/flash-attention>

## 13.6 Multi-Query Attention

Paper:

[Fast Transformer Decoding: One Write-Head is All You Need](#)

## 13.7 Grouped-Query Attention

Paper:

[GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints](#)

## 13.8 Transformer-XL

Paper:

[Transformer-XL: Attentive Language Models Beyond a Fixed-Length Context](#)

## 13.9 RoPE

Paper:

[RoFormer: Enhanced Transformer with Rotary Position Embedding](#)

## 13.10 ALiBi

Paper:

[Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation](#)